CISC 322/326
A1: Conceptual Architecture Report
October 9, 2026

Conceptual Architecture of WezTerm

Group 11: g(ai)ls

Authors

Aimee Anwar - 23cj68@queensu.ca
Manreet Gill - manreet.gill@queensu.ca
Akshay Kapal - akshay.kapal@queensu.ca
Maria Koukharsky - 23xht3@queensu.ca
Sophie Liang - 23xgt4@queensu.ca
Ava Nikolaus - ava.nikolaus@queensu.ca

Table of Contents

1 Abstract **3**
2 Introduction and Overview **3**
3 Architecture **3**
3.1 Overall Structure 4
3.2 Architectural Styles 5
3.3 Component Descriptions 6
3.4 Control Flow and Concurrency 7
3.5 Critical Abstractions and Patterns 8
3.6 Alternative Architectural Styles 8
4 Derivation Process **8**
5 External Interfaces **9**
6 Use Cases **10**
6.1 Use Case 1 10
6.2 Use Case 2 11
7 Data Dictionary **12**
8 Naming Conventions **13**
9 Conclusions **14**
10 Lessons Learnt **14**
11 References **15**
12 AI Collaboration Report **15**
12.1 AI Member and Selection Process 15
12.2 Tasks Assigned to the AI Teammate 16
12.3 Interaction Protocol and Prompting Strategy 16
12.4 Validation and Quality Control Procedures 17
12.5 Quantitative Contribution to Final Deliverable 17
12.6 Reflection on Human-AI Team Dynamics 17

1 Abstract

WezTerm is an open-source, GPU-accelerated, cross-platform terminal emulator and multiplexer written in Rust. It lets developers run shells in tabs and split panes, keep sessions alive locally or on remote machines, and customize nearly everything through a Lua configuration file. A substantial issue with traditional terminals is that they leave session management to a separate tool such as tmux, which splits the experience across programs. WezTerm addresses this by combining terminal emulation, multiplexing and scripting in one application. Its conceptual architecture has eight components. The CLI Launcher, GUI Frontend, Font and Appearance, and Configuration and Scripting Engine handle the user-facing side, while the Multiplexer, Terminal Emulation Core, PTY and Process Layer, and Mux Server and Transports manage terminal sessions, local or remote. There are two styles that characterize the design. The first being client-server which lets sessions outlive a disconnected client, at the cost of latency and security configuration. The second being event-driven, which lets users extend WezTerm through Lua handlers without modifying it but makes execution order harder to trace. There are two use cases that are explained with the sequence diagrams. The cases include pressing a bound or unbound key and connecting over SSH. The modular design makes it easier for WezTerm to add new features such as transports and rendering back ends over time.

2 Introduction and Overview

This report will delve into the conceptual architecture of WezTerm. The purpose of this document is to deconstruct the architectural styles, structural patterns, and concurrency decisions that are present in the software. The analysis focuses on the documentation of the project, specifically how WezTerm processes user input, interprets terminal escape sequences, communicates with shell processes through pseudo-terminals, and uses the GPU to render text.

The scope covers WezTerm's software components and how they interact with the immediate external environment, which includes various elements of the user interface, such as handling user commands, changing appearance, and more. Additionally, the internal separation between terminal-state modelling, multiplexing, display, as well as the local and remote multiplexor modes are all within scope. Unlike a simple terminal that only displays shell output, WezTerm acts as a session manager that provides tabs and workspaces, along with a mux server that lets these sessions remain active and be accessed remotely.

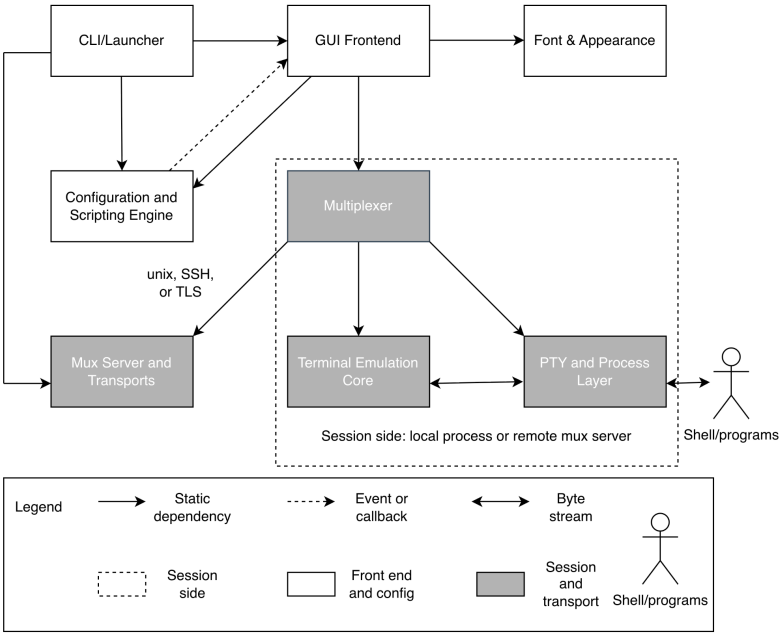
3 Architecture

3.1 Overall Structure

WezTerm is a terminal emulator and multiplexer. It creates a pseudo-terminal (PTY), starts a shell in it, interprets the escape sequences the program emits to draw a display, and encodes keyboard and mouse input as escape sequences for the program [1]. We organize it into eight components: CLI Launcher, GUI Frontend, Configuration and Scripting Engine, Font and Appearance, Terminal Emulation Core, Multiplexer, Mux Server and Transports, and PTY and Process Layer. Figure 1 shows them and their dependencies, and Table 1 explains each arrow ("A

→ B" means A relies on B's services). Shell/programs is an external system that WezTerm launches and communicates with, not a WezTerm component.

Figure 1:



The multiplexer is a separate server that can run without a GUI. The color palette is on the multiplexer server for SSH and TLS domains because it is a part of the terminal emulation state. So, the Multiplexer, Terminal Emulation Core, and PTY & Process Layer form a session side that the GUI renders as a client. For a local domain, we infer the session side runs inside the GPU process. This is supported by the contributing guide's advice to start wezterm-gui with --always-new-- process so mux logs aren't hidden in the background.

Table 1: Dependencies

Dependency	What it represents	Interface
CLI Launcher → GUI Frontend	wezterm delegates to wezterm-gui when a GUI is needed	Process launch and arguments
CLI Launcher → Configuration	Options choose or override the config	CLI options
CLI Launcher → Mux Server and Transports	connect, ssh, serial, and cli establish connections	connection requests
GUI Frontend → Configuration	Reads settings and key bindings, and is notified of reloads	config values
Configuration → GUI Frontend	Lua handlers receive window and pane objects and can trigger actions	Lua event callbacks
GUI Frontend → Font and Appearance	Needs glyphs, colors, and tab bar styling to draw	glyph and color requests
GUI Frontend → Multiplexer	Attaches a domain's windows and tabs to the native UI	domain attach/spawn requests
Multiplexer → Terminal Emulation Core	Each pane holds terminal state	screen state and input events
Multiplexer → PTY and Process Layer	Multiplexer manages running programs, it spawns them through a PTY	spawn requests
Multiplexer → Mux Server and Transports	Remote domains reach a remote multiplexer	Unix socket, SSH, or TLS channel

Terminal Emulation Core ↔ PTY and Process Layer	Program output comes in, encoded input goes out	PTY byte stream
PTY and Process Layer ↔ Shell/programs	The PTY connects WezTerm to the program it spawned	PTY byte stream

3.2 Architectural Styles

Client-Server: A GUI client connects to a multiplexer server over a Unix socket, SSH, or TLS. The server keeps the terminal state, which is why a remote domain's color scheme comes from the server's config.

Benefits: With the session maintained by the server, it can continue even when the client disconnects. So, in practice, if a TLS connection was interrupted, WezTerm can reconnect and continue the existing session. The same server architecture also allows for multiple transport mechanisms with additional ones being introduced over time. The documentation's "Since" annotations indicate that Unix sockets, SSH, and TLS were added in different releases. Windows and tabs belonging to a domain also benefit from the native UI's handling of mouse input, clipboard operations, and scrollback.

Challenges and repercussions: Network latency becomes an important consideration with this architecture, so WezTerm provides a predictive local echo, which activates when the round-trip latency exceeds a configurable threshold. However, this feature increases the complexity of client. SSH domains also require a compatible version of WezTerm to be installed on the remote host. Security configuration becomes more involved as well: TLS uses SSH during bootstrapping to obtain a certificate, while Unix sockets are protected through ownership checks. Session persistence is optional, and ordinary WezTerm ssh sessions terminate if the network connection is lost.

Event-driven (implicit invocation): Key presses are compared against a table of key bindings and then dispatched to their corresponding named actions. A LEADER key can also alter which bindings are recognized. Event handlers register for named events using wezterm.on, with those events originating from either WezTerm itself or from user-defined code. Lifecycle events are available at both architectural layers: mux-startup is triggered when the multiplexer server starts, whereas gui-startup is triggered when the GUI starts. Changes to the configuration also cause it to reload.

Benefits: Event-driven architecture allows users to extend WezTerm's behavior without actually changing WezTerm. Event handlers can define custom events, while plugins can be loaded directly from Git URLs. Event emitters do not need to know which listeners will respond to them, which we infer reduces coupling between components. Configuration reloads also recreate the Lua state and clear registered handlers, making hot reloading relatively straightforward.

Challenges and repercussions: The order that the code executes can be easy to overlook. Event handlers execute in registration order, and if a handler returns false, it can prevent subsequent handlers from running or suppress a default action. Registered handlers also cannot be de-registered. Window creation and process spawning are asynchronous operations that cannot be awaited, forcing scripts to approximate synchronization using fixed delays. Additionally, the

configuration file may be evaluated multiple times, meaning that side effects performed at file scope, such as spawning windows, can accumulate across evaluations. Modal states (i.e. LEADER) consume other key presses until the state times out. Plugins are not automatically updated and, as we infer, involve executing third party code.

3.3 Component Descriptions

CLI Launcher: The CLI Launcher component provides the command-line entry point. wezterm is intended for scripting and delegates to wezterm-gui when needed, while wezterm-gui is better suited for desktop launchers, especially on Windows, as it avoids opening a console window. Subcommands such as start, ssh, connect, and cli, along with global options, determine the behavior and configuration

GUI Frontend: The GUI Frontend displays sessions in native windows and handles user input. It manages windows, tabs, split panes, mouse selection, bracketed paste, and hyperlinks, while mapping key presses to actions. It supports three rendering backends: OpenGL, WebGpu (Metal, Vulkan, or DirectX 12), and Software, which is automatically selected during Windows Remote Desktop sessions. Rendering is likely performance-critical.

Configuration and Scripting Engine: The Configuration and Scripting Engine allows users to customize WezTerm using Lua 5.4. It searches for configuration files in a fixed order, falls back to built-in defaults if loading fails, and reloads when the file changes or CTRL+SHIFT+R is pressed. Command-line and per-window overrides take precedence over file settings. It also manages key bindings, domain definitions, event handlers, and plugins, making it a dependency for most other components.

Font and Appearance: Converts text and appearance settings into glyphs and colors. It selects glyphs using an ordered font fallback list, ending with a “Last Resort” placeholder, and applies color schemes according to documented precedence rules. It also controls tab bar styling, inactive pane dimming, background images, gradients, and opacity.

Terminal Emulation Core: Maintains terminal state and processes program output. It interprets ECMA-48/ANSI escape sequences, supports iTerm2, Kitty, and Sixel image protocols, manages a color palette that escape sequences can modify at runtime, and maintains per-tab scrollback with a search overlay. It also encodes keyboard and mouse input for running programs. The contributing guide describes the terminal model as independent of the windowing system, designed for xterm compatibility, and supported by terminal testing helpers.

Multiplexer: The multiplexer organizes running programs into panes, tabs, windows, and workspaces, even without an attached GUI. A default local domain is created at startup, and new processes can be spawned in the default, current, or a named domain.

Mux Server and Transports: This component hosts the remote multiplexer and manages client connections. The wezterm-mux-server runs as a background process, including within WSL 1, and supports Unix sockets, SSH domains (libssh2), and TLS over TCP established through SSH, with automatic reconnection. The separate wezterm ssh command provides non-persistent connections, opening tabs as channels within a single session. Serial port connections cannot create new tabs, suggesting they bypass the mux server.

PTY and Process Layer: Abstracts operating-system differences when running programs. It creates a PTY and launches a shell on the client side, using kernel PTYs on Unix and ConPTY with a helper process (openconsole.exe, sometimes conhost.exe) on Windows. WezTerm supports Linux, macOS, Windows 10, FreeBSD, and NetBSD.

3.4 Control Flow and Concurrency

On startup the GUI program marks the main thread before calling run. After running, the program parses the command line, establishes the environment, loads the Lua configuration, and picks a subcommand (start, connect, ssh, serial, ls-fonts, show-keys3). For start, the program searches for a matched active GUI; if one is found, it connects to the socket, spawns a tab then exits. If a running GUI cannot be named, the program constructs the front end and queues an asynchronous task before calling run forever which passes the thread to the message loop.

The first file found in the configuration path runs as a Lua script synchronously through a blocking call. Upon call, a validated table containing setting structures and defaults are returned. If the first file found is broken, it keeps the current setting and stores the error to display when the frontend is running. After analyzing, command-line changes are updated, settings are validated and defaults are filled. Every load builds a new Lua context resulting in each handler being registered from empty. A backend thread also watches the files and starts a reload once changes are applied, at the end the new state is passed to the main through a channel.

Following startup, an initial task invokes the gui-startup event and constructs a multiplexer window. The multiplexer notifies the frontend that the window has been added and assists by creating a matching GUI on the main thread. Allowing the window to exist before the first tab. The tab is then added to trigger the gui-attached event, a guard is built in to prevent the program from exiting early when it runs. Once the GUI window is constructed, it figures out its size from its tab, connects to the screen display then runs the events.

All windows events, including physical input are dispatched in one function on the GUI thread. When a key is pressed, the window verifies if it is the leader key, if an active interface wants it, and if it is key binding. If a binding matches, the action runs and a binding with a Lua callback runs its function, otherwise the key is sent to the active pane. Two handlers work with keyboard input; one matches keys with bindings and the other compares against window and default tables. If no binding, temporary key tables with planned expiry on unknown input are cleared before it is sent to the pane. Every pane has two background threads, a reader thread that conducts blocking reads and a parser thread that analyzes escapes to group output together and attach the result. The main thread communicates with the window to validate if the pane is visible. A repaint call runs the paint routine which draws all aspects of the frame including, backgrounds and borders. Once the shell exits, exit behaviour determines if the pane remains open or closed.

Finally, the Lua handlers run on the main thread asynchronously in a queue that allows a running and waiting event per window. Handlers of events run the order in which they registered. The only exception to this is the window title as it runs immediately and uses the first handler. Scripts will never adjust the window directly, only send messages and waits for a reply when questioned. Events that follow this structure include status updates that are timed for repeats and origins of

Lua callbacks. When the last pane is closed, the multiplexer records that there is nothing left to run and the frontend concludes when no activity is present.

3.5 Critical Abstractions and Patterns

WezTerm utilizes abstraction and design patterns as a foundation for its structure. The core abstraction is the multiplexer; domains assist by determining where a pane will run and ensuring the system is able to support local and remote runs. The system is event-focused and uses an Observer pattern to communicate adjustments between components. The multiplexer synthesizes notifications for events like pane creation by the domain that components can retrieve. WezTerm uses a proxy pattern to illustrate remote panes and domains locally, this allows for interactions of interfaces with terminal sessions. In addition, WezTerm follows a client-server architecture that enables the multiplexer to work separately from the graphical interface.

3.6 Alternative Architectural Styles

In a layered architecture style, the system would be distributed into distinct layers, with each one in charge of a specific function and communicating with those near it. If WezTerm implemented this structure, the graphical interface would be the top layer while the terminal, multiplexer and operating system layers would follow. This style improves organization and management as each layer has an identified task which allows components to be modified without the system being changed. In contrast, this structure may restrict concurrency because communication travels between layers. The trade-off of a layered structure is increased distribution between responsibilities in exchange for reduced efficiency.

The WezTerm architecture could also follow a pipe-and-filter style where the system would be a group of independent transformations presented by different components. The terminal data could be envisioned as a flow through processing stages. In addition, a program running in a pane produces output via PTY which is passed to the parser. The parser would then analyze the data and update status to display output. This architecture could support concurrency by promoting the processing of data in different steps simultaneously. In comparison, as WezTerm's current style is event driven, a pipe-and-filter implementation could produce overhead as data is passed between processing. This is a result of non-direct communication through a pipe system opposed to a direct response to events. Therefore, the potential for greater concurrency in the system comes at the risk of communication overhead.

4 Derivation Process

The architecture of WezTerm was derived by conducting research to understand its use cases and functionalities. The first step was analyzing the available documentation on WezTerm's capabilities and features [5], along with the course material on onQ [3]. On a surface level, it is evident that WezTerm is designed to be used by developers who require a complex, controllable terminal environment. It is set apart from other terminal environments due to its configuration using the Lua scripting language. WezTerm has built-in features to allow for splitting panes, managing tabs, and detaching/reattaching concurrent background sessions without requiring a separate daemon through local and remote hosts.

From establishing WezTerm's core usages and using the documentation on WezTerm, we find that it is configured using eight components that were previously mentioned. To provide an entry point for users to interact with the software, the developers implemented the CLI Launcher component which interprets user commands. For the user to be able to see the graphical items that they may be managing, the developers implemented a GUI front-end. The configuration component searches for a custom Lua configuration file and falls back to the default if none is found. The terminal emulation core creates a PTY with a spawned child program, interprets escape sequences to produce displays, and decodes keyboard and mouse input before re-encoding it as escape sequences for the running program. The multiplexer was implemented to allow users to create domains which manage the windows and tabs of the user interface. To allow users to create remote hosts, the multiplexer servers and transports enable connections to a SSH domain, Unix domain, and TLS domain. The software also required a bridge between the userspace and kernel, which led to the implementation of the PTY and process layer component. These key components demonstrate that the developers were primarily focused on streamlining the user interface of the terminal environment and allowing for flexibility in the terminal configuration.

Outside resources, such as videos demonstrating WezTerm's functionality [2, 4], were then reviewed to identify realistic use cases and to see how the components interact. The two most notable use cases were pressing a bound or unbound key, and successfully or unsuccessfully connecting to a remote host over SSH. These use cases were developed into sequence diagrams, and the components found in the sequence diagrams were used to create a box-and-arrow diagram. The components appearing in the sequence diagrams were then combined into the box-and-arrow diagram in Figure 1. The diagram shows the static dependencies between components, and its dotted arrows show event and callback flow. It focuses on how the front-end and configuration components work with the session and transport components. The diagram provides a summary on the interactions between the components that make up the architecture of WezTerm.

5 External Interfaces

User Interface
The user interface is the terminal environment where users can interact with the software.

- Input: Handled by CLI Launcher
    - e.g. user commands (start, ssh, serial)
- Output: Handled by GUI frontend.
    - e.g. creating new windows, displaying a font, changing colour scheme

Multiplexing Domains
The multiplexing domains are connections made to a remote WezTerm multiplexer.

- SSH Domains: connection made via a ssh connection
- Unix Domains: connection made via a Unix socket
- TLS Domains: connection made via TLS encrypted TCP connection

Serial Connections

WezTerm can connect to serial ports as a client.

- Serial Ports: used to establish a serial connection (e.g. working with embedded devices, connecting to a serial console on a headless server)

Pseudo-Terminal teletYpe (PTY)
The PTY provides a connection between the user space and the kernel.

- Controller side: passes information about terminal's user interface (e.g. window size)
- Client side: acts as an I/O stream (e.g. vim)

Shell Integration (Operating System Command Escape Sequences)
The shell can be integrated with WezTerm through Operating System Command (OSC) escape sequences.

- OSC 7: escape sequences that advise the terminal of the working directory
- OSC 133: escape sequences that define zones for input, output, and prompt
- OSC 1337: escape sequences that set user vars to track additional shell state

6 Use Cases

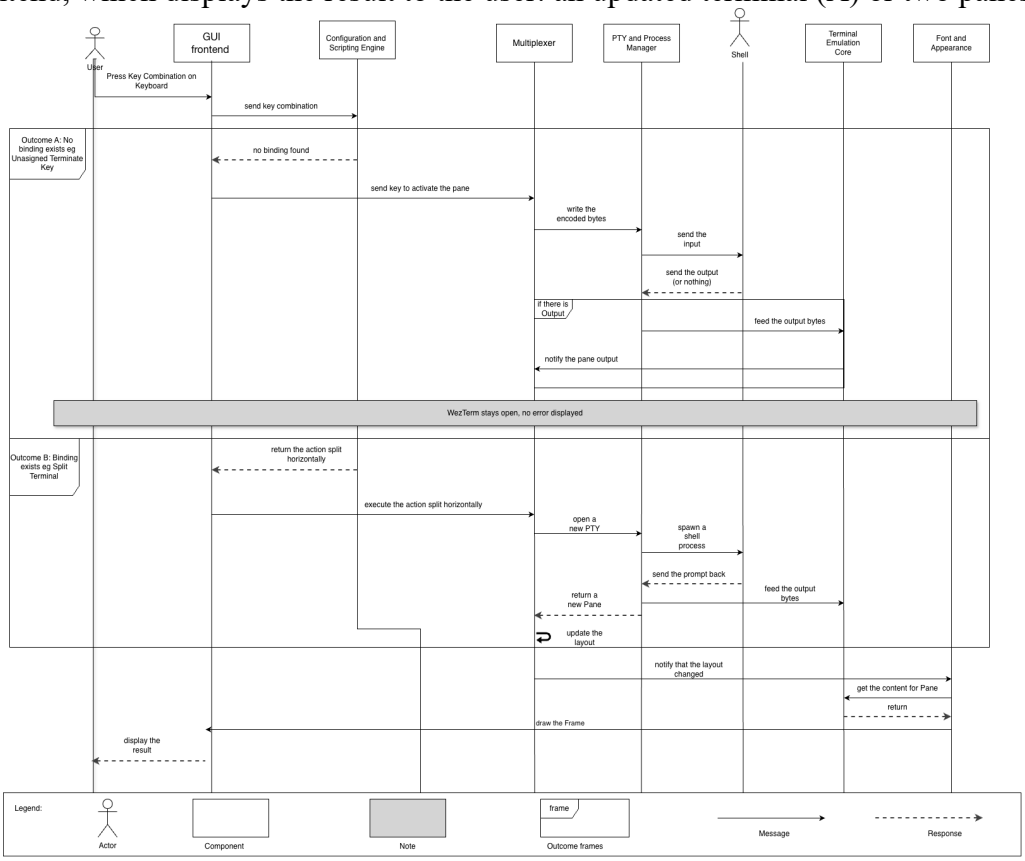
6.1 Use Case 1

Use Case 1: Key Bindings (Unbound vs. Bound Key): The user presses key combinations that either do or do not have a binding in WezTerm. Phase one is the input reception. The user presses a key combination on the keyboard, and the GUI frontend receives it. Phase two is the binding lookup. The GUI frontend sends the key combination to the Configuration and Scripting Engine, which checks whether it matches an entry in the key table. The key table is built from WezTerm's default bindings plus any the user added in their Lua configuration file. The Configuration and Scripting Engine returns:

Outcome A: No binding exists. For example, the user presses a combination they expect to terminate WezTerm, but none is assigned. The Configuration and Scripting Engine responds to the GUI frontend that no binding was found. WezTerm does not close, because it never recognized the key as a command. The GUI frontend sends the key to the Multiplexer to activate the pane. The Multiplexer writes the encoded bytes to the PTY and Process Manager, which sends the input to the Shell. The Shell sends back its output, or nothing at all. If there is output, the PTY and Process Manager feeds the output bytes to the Terminal Emulation Core, which notifies the Multiplexer of the pane output. Either way, WezTerm stays open and displays no errors.

Outcome B: A binding exists. For example, the user presses a key assigned to splitting the terminal horizontally. The Configuration and Scripting Engine returns the split horizontally action to the GUI frontend, and the GUI frontend asks the Multiplexer to execute it. The Multiplexer asks the PTY and Process Manager to open a new PTY, and the PTY and Process Manager spawns a shell process. The Shell sends its prompt back to the PTY and Process Manager, which feeds the output bytes to the Terminal Emulation Core and returns the new Pane to the Multiplexer. The Multiplexer then updates the layout.

Phase three is the render phase, which both outcomes share. The Multiplexer notifies Font and Appearance that the layout changed. Font and Appearance gets the content for the Pane from the Terminal Emulation Core, which returns it. Font and Appearance then draws the frame to the GUI frontend, which displays the result to the user: an updated terminal (A) or two panes (B).

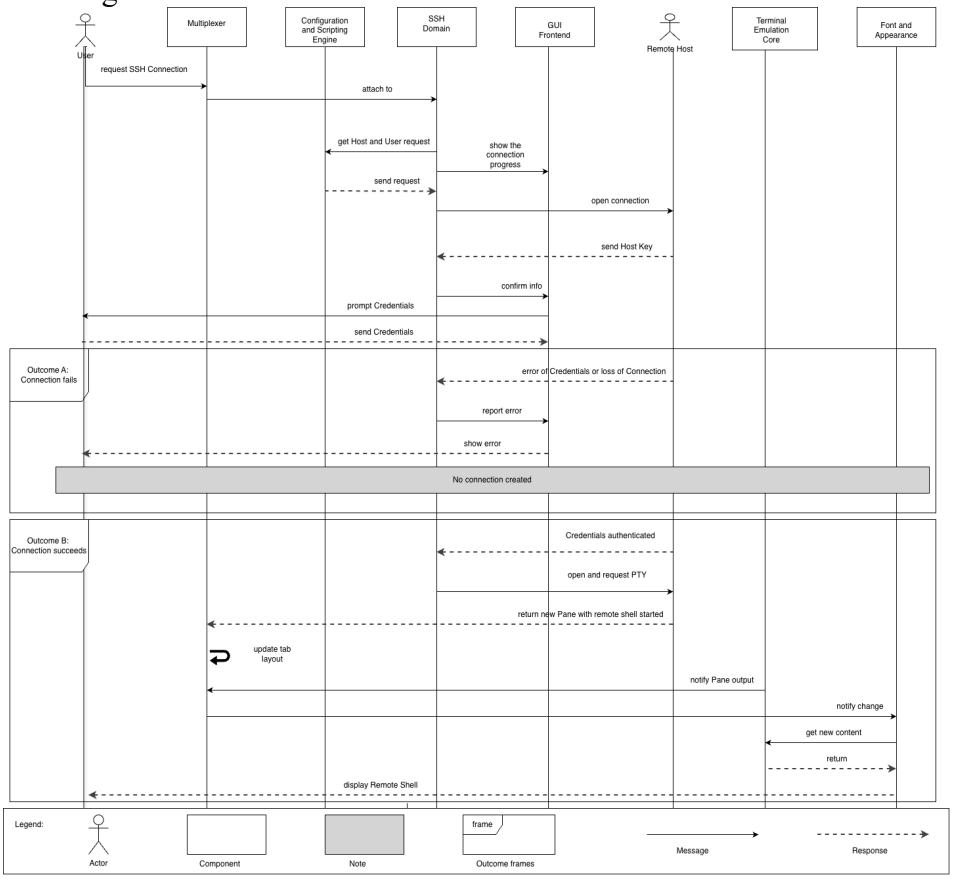


6.2 Use Case 2

Use Case 2: Connecting to a Remote Host over SSH (Success vs. Failure): The user wants to connect to a remote machine using WezTerm's built-in SSH client. Phase one is the connection request. The user requests an SSH connection, and the Multiplexer attaches to the SSH Domain. The SSH Domain asks the Configuration and Scripting Engine for the host and user request, which the engine sends back. While the SSH Domain connects, it tells the GUI frontend to show the connection progress to the user, then opens a connection to the Remote Host. Phase two is the handshake and authentication. The Remote Host sends its host key to the SSH Domain, which asks the GUI frontend to confirm the information. The GUI frontend then prompts the user for credentials, and the user sends them back to the GUI frontend. The connection then ends in one of two outcomes:

Outcome A: The connection fails. The Remote Host sends the SSH Domain an error, either because of bad credentials or because the connection was lost. The SSH Domain passes the Error to the GUI frontend, which shows it to the user. No connection is created, and the existing tabs and panes are unaffected. The user can then correct the problem (fix the hostname, supply the right credentials) and try again.

Outcome B: The connection succeeds. The Remote Host confirms that the credentials were authenticated, and the SSH Domain opens a channel and requests a PTY on the Remote Host. The remote shell starts, and a new Pane is returned to the Multiplexer, which updates the tab layout. The Terminal Emulation Core notifies the Multiplexer of the Pane output (the shell prompt). The Multiplexer then notifies Font and Appearance of the change. Font and Appearance gets the new content from the Terminal Emulation Core, which returns it, and the user sees the remote shell displayed. From then on, keystrokes go to the Remote Host instead of a local shell, but the key-handling flow is the one from use case 1.



7 Data Dictionary

This is a glossary of the key terms used in this architectural report.

Term	Definition
Client-server	A GUI client connects to a mux server that holds the terminal state, so sessions can outlive a disconnected client and reconnect.
ConPTY	The Windows pseudo-console, which plays the role of the Unix PTY.
Domain	A group of windows, tabs and panes tied to a place where they run: the local machine or a remote multiplexer reached over SSH, a Unix socket or TLS.
Escape sequence	Special bytes in program output that tell the terminal how to draw, such as changing colour or moving the cursor. WezTerm also encodes keyboard and mouse input as escape sequences for the program.

Event-driven (implicit invocation)	Components react to named events instead of being called directly. Key presses are dispatched to actions, and Lua handlers respond to events such as gui-startup.
Key binding	A rule, from the defaults or the user's Lua file, that maps a key combination to a named action such as splitting a pane.
Lua configuration	The Lua script that configures WezTerm ("code-as-configuration"), loaded at startup and reloaded when it changes.
Lua event handler	A Lua function registered with wezterm.on that runs when a named event occurs, such as gui-startup or mux-startup.
Mux server	A multiplexer daemon that can run without a GUI and keeps terminal state while clients connect and disconnect.
Observer pattern	The Multiplexer sends notifications, such as a new pane being created, to components that subscribe to them.
OSC (Operating System Command)	Escape sequences used for shell integration, such as OSC 7 (working directory) and OSC 133 (prompt, input and output zones).
Pane, tab and window	A window holds tabs and a tab holds panes. A pane is a view of one independent terminal session, tracking its PTY process and screen content.
Pane threads	Each pane has a reader thread that does blocking reads from the PTY and a parser thread that interprets the escape sequences.
PTY (pseudo-terminal)	A channel between user space and the kernel. The controller side carries information such as window size, and the client side is the I/O stream used by the program.
Screen state and scrollback	The terminal's current contents (characters, colours, cursor) and the saved lines that have scrolled off.
Shell	A command interpreter such as bash or zsh, run inside a pane. Not part of WezTerm.
Terminal emulator	A program that starts a shell in a pseudo-terminal, interprets the escape sequences it emits to draw a display, and turns input into escape sequences for it.
Transport	The connection type between WezTerm and a remote session: SSH, Unix socket or TLS. Serial ports are also supported as a client connection.

8 Naming Conventions

1- Abbreviations:

API	Application Programming Interface
CLI	Command Line Interface
GPU	Graphics Processing Unit
GUI	Graphical User Interface
UI	User Interface
OS	Operating System

PTY	Pseudo-Terminal
ConPTY	Windows Pseudo Console
Mux	Multiplexer
SSH	Secure Shell
TLS	Transport Layer Security

9 Conclusions

This report analyzed WezTerm's conceptual architecture and found a client-server style with elements of an event-driven style. Its components are the CLI, GUI, configuration and scripting engine, user interface, terminal emulation core, multiplexer, mux servers, PTY, and process layer.

The client-server style lets a mux server maintain terminal state independently of the GUI, so sessions can outlive a disconnected client and reconnect. The event-driven style supports extensibility, letting users change WezTerm's behaviour and appearance without modifying it. Window events and Lua handlers are processed on the main thread. The use cases show these styles working together: in the key-binding case, one input path either forwards a key to the shell or triggers an action such as splitting a pane, and in the SSH case, a failed connection is reported through the GUI without affecting existing tabs, while a successful one hands a new pane to the multiplexer. The analysis also highlighted trade-offs. Because terminal state lives on the server, local configuration cannot control every aspect of a remote session. Network latency is mitigated with local echo. SSH domains require a compatible WezTerm version on the remote host, TLS and Unix socket domains add security setup, and Lua configuration is powerful but harder to learn than a static format.

Overall, WezTerm favours flexibility over minimalism. At the cost of greater complexity, it acts as a session manager with persistent, remotely accessible sessions rather than just a terminal that displays shell output.

10 Lessons Learnt

A use case is a goal
We first treated each outcome as its own use case. We learned that one use case, like "press a key" or "connect over SSH," includes the main success path and the ways it can go wrong or branch, which we show as outcome frames in one sequence diagram. This also showed us how much can go wrong in even a simple interaction: an unbound key, a rejected host key, a refused login, an unreachable host. Next time we would pick two use cases that exercise different parts of the architecture, then list the alternative outcomes for each before drawing.

Agree on component names before anyone writes

We ended up with three naming conventions: the architecture boxes, the sequence diagram participants, and the naming conventions draft. Matching them cost time and risked inconsistency marks. We would freeze one official names list as soon as the box diagram exists.

A plan with clear task ownership prevents rework
Our section split gave every part an owner, but some sections depended on others. The use cases and component descriptions both depended on the final box diagram. Because we didn't lock the order early, sections were written with placeholder names that later had to be reconciled. Next time we would fix the dependencies first: diagram, then names, then everything else.

Humans must stay in the loop
The AI saved time on drafts and reviews, but it sometimes sounded confident about details it had not verified, and its component list came from memory. Every claim about WezTerm had to be checked against the source, the docs or the course material by a teammate. The AI helped us think, but the group had to decide. Next time we would log AI use from day one because reconstructing prompts, errors caught and rough percentages after the fact is slow and less credible. A shared log from the start would have made the AI report much easier.

11 References

Adams, B. (n.d.) *SRS example as seen in class* [PDF Document]. OnQ. https://onq.queensu.ca/d2l/le/content/1199240/viewContent/7022727/View

Better Stack. (2025 September 8). *Why WezTerm Beats Ghostty, Warp, and tmux COMBINED*. [Video]. YouTube. https://www.youtube.com/watch?v=kYBvkzwM8Iw

Hassan, A. E., & Adams, B. (n.d.). *Module 03: Architecture styles* [PowerPoint slides]. onQ. https://onq.queensu.ca/d2l/le/content/1199240/viewContent/7022689/View

Josh Medeski. (2023, December 14). *More fun in the terminal with Wezterm!* [Video]. YouTube. https://www.youtube.com/watch?v=hqP0XdB7VcA

WezTerm. (n.d.). *WezTerm documentation*. https://wezterm.org/

12 AI Collaboration Report

12.1 AI Member and Selection Process

Name: Claude, **Version:** Claude Sonnet 5.5, used through the claude.ai chat interface with web-search-on from September 26th, 2026, to October 9, 2026.

Selection process and justification: Our team compared Anthropic's Claude Sonnet 5.5 (free tier) with OpenAI's ChatGPT (free tier, GPT-5.6 Luna). While both are capable general-purpose assistants, we selected Claude Sonnet 5.5 for three main reasons:

1. Our delegated work consisted mainly of reviewing architectural diagrams and auditing drafts against the assignment rubric. These are text-reasoning tasks that suit a general-purpose language model, and both tools were technically adequate for them.

2. Our workflow depended on uploading PDFs, such as the project guidelines, alongside screenshots of our rubric and our diagrams, and on iterating on them many times in a short period. Although ChatGPT's free tier allows unlimited text chat, we repeatedly reached its limit on uploads and had to wait several hours before we could continue. These interruptions made it impractical for our workflow. Claude's free plan allowed us to keep working through these review cycles.
3. Claude accepts long pasted context and images, so we could supply the guidelines, rubric, written use cases, and diagram screenshots within a single conversation and keep working without restating the background each time.

However, we were aware of Claude's limits from the beginning. Its knowledge of WezTerm derives from training data and may be inaccurate, it retains context only within a single conversation, so we treated everything it said about WezTerm as unverified and relied on WezTerm documentation for significant details.

12.2 Task Assigned to the AI Teammate

- **Technical Articulation:** The AI Teammate explained what a terminal emulator and multiplexer are and suggested candidate components and architectural styles. This was suitable for the AI because it gave us vocabulary and a starting structure to react to.

- **Diagram and Rubric Review:** We gave the AI Teammate our completed box-and-arrow diagram, our sequence diagrams and our written use cases, along with the rubric, and asked it to find mismatches. It caught problems such as a swapped colour legend, a missing event arrow and diagram steps that did not match our text. This was suitable because of the AI's context window capability. It was able to hold the entire rubric in memory, allowing it to spot inconsistencies that a human reviewer may miss.

12.3 Interaction Protocol and Prompting Strategy

Methodology & Workflow: Each member used their own chat and recorded every interaction in a shared log (ex. using Threads on Discord for screenshots of chat prompts + AI's responses). Each interaction followed the same steps: define the objective, assemble the context, run the prompt, and review the result as a group. We treated the AI teammate as a drafting and review assistant and checked its output against course materials before using it.

Prompting Strategy & Iteration: We gave context first, including the project guidelines, the rubric and links to the WezTerm documentation. We used technical vocabulary and short explanations of how the open-source project works for anything that the AI teammate missed in its initial understanding of WezTerm, so that the AI did not have to guess what we meant. We also asked for specific formats, such as a table or a fixed word count. The answers we received early-on were often too broad, so we added missing context and narrowed the question instead of restarting the chat.

Example Prompt: "Act as a Professional Solutions Architect. Knowing this is what I am trying to represent, are the following two use case diagrams good? Point out any flaws in logic or arrows that may be redundant or not specific." We attached both use case diagrams.

12.4 Validation and Quality Control Procedures

Our validation process involved cross-checking each claim rather than passive review. We verified the AI's statements about WezTerm's structure against the official documentation. We compared its descriptions of architectural styles with the course architecture-styles slides. For diagrams, every sequence diagram participant had to be a box in our architecture diagram, and every arrow had to match a dependency there. A teammate other than the author reviewed each AI-assisted section, and we rewrote AI wording in our own words.

12.5 Quantitative Contribution to Final Deliverable

We estimate that the AI contributed approximately 30% to the final deliverable. This figure comes from the fact that the AI's contribution was mainly analytical and editorial, such as critique and drafting support, while the architecture itself was developed and decided by the team.

12.6 Reflection on Human-AI Team Dynamics

Assimilating an AI teammate into our group was helpful for working efficiently, and making well-thought-out decisions, rather than second-guessing ourselves. More specifically, it played a significant role in the early stages of our report by explaining technical terminology such as what a terminal multiplexer is, as well as brainstorming alternative architecture styles. Later on, it played a role as a reviewer that caught naming and logic mismatches, we had missed and any major inconsistencies in our report against the rubric. However, our teammate also created additional work for us. We had to verify its claims, rework our diagrams in response to its feedback, and keep refining our prompts. It often confidently hallucinated details about our report draft it had not checked, which taught us to confirm every output against what our group members wrote.

Additionally, working on a free plan added a practical limit, since uploads and long conversations used up our allowance, we planned our prompts ahead and saved them for the highest-value tasks, such as full diagram reviews. Due to this, we deliberately kept certain responsibilities with the team: the final architecture decisions, checking claims against the WezTerm documentation, the derivation process, and our own reflections. These had to reflect what we did and understood, and the AI could not stand in for that. This also affected our decision-making, since the AI proposed the architectural styles, we later had to defend, so we made sure the group discussed and agreed on them instead of adopting them by default.

Throughout this process, our group learned that AI is most effective as a critic and organizer of our own ideas and not as a source of truth, and that the best workflow was to give it context and ask it to audit our reasoning. In future projects, we would also start logging every interaction early-on, which would have made this report much easier to write!